

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 3**



Linked List

Disusun oleh :
Vivaldi Fachmy Fahlevi **NIM.2510817210017**

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 3

Laporan Praktikum Algoritma Dan Struktur Data Modul 3 : Linked List ini disusun sebagai syarat lulus mata kuliah Praktikum Basis Data I. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Vivaldi Fachmy Fahlevi
NIM : 2510817210017

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir.Muhmmad Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL.....	5
SOAL 1	1
A. Source Code	1
B. Output Program	12
C. Pembahasan	13
SOAL 2	20
A. Source Code	22
B. Output Program	25
C. Pembahasan	26
SOAL 3	29
A. Source Code	29
B. Output Program	40
C. Pembahasan	41
SOAL 4	43
A. Source Code	45
B. Output Program	50
C. Pembahasan	51

DAFTAR GAMBAR

Gambar 1 Screenshot Output Program Soal 1	12
Gambar 2 Screenshot Output Program Soal 2	25
Gambar 3 Screenshot Output Program Soal 3	40
Gambar 4 Output Program Soal 4.....	50

DAFTAR TABEL

Tabel 1 Source Code File single_linked_list.h	1
Tabel 2 Source Code File single_linked_list.cpp.....	2
Tabel 3 Source Code File test_single.cpp	10
Tabel 4 Source Code File prob_a.cpp	22
Tabel 5 Source Code File double_linked_list.h	29
Tabel 6 Source Code File double_linked_list.h	30
Tabel 7 Source Code File test_double.cpp.....	38
Tabel 8 Source Code File prob_b.cpp.....	45

SOAL 1

Implementasi Senarai Berantai Tunggal Melingkar

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`. Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap. Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 1 Source Code File `single_linked_list.h`

1	<code>#ifndef SINGLE_LINKED_LIST</code>
2	<code>#define SINGLE_LINKED_LIST</code>
3	
4	<code>struct Node {</code>
5	<code> int data;</code>
6	<code> Node* next;</code>
7	<code>};</code>
8	
9	<code>struct SingleLinkedList {</code>
10	
11	<code> Node* head = nullptr;</code>
12	<code> Node* tail = nullptr;</code>
13	
14	<code> int size = 0;</code>
15	
16	<code> void init();</code>
17	<code> bool is_empty();</code>
18	
19	<code> void add_front(int inputValue);</code>

20	void add_back(int inputValue);
	void add_idx(int inputValue, int
21	indexPosition);
22	
23	void delete_front();
24	void delete_back();
25	void delete_idx(int indexPosition);
26	
27	void display();
28	
29	int get(int indexPosition);
30	
31	void set(int inputValue, int indexPosition);
32	
33	void clear();
34	};
35	
36	#endif

Tabel 2 Source Code File single_linked_list.cpp

1	#include "single_linked_list.h"
2	#include <iostream>
3	
4	using namespace std;
5	
6	void SingleLinkedList::init() {
7	
8	head = nullptr;
9	tail = nullptr;
10	
11	size = 0;
12	}

```

13
14 bool SingleLinkedList::is_empty() {
15
16     return head == nullptr;
17 }
18
19 void SingleLinkedList::add_front(int inputValue)
20 {
21     Node* newNode = new Node;
22
23     newNode->data = inputValue;
24     newNode->next = nullptr;
25
26     if (is_empty()) {
27
28         head = newNode;
29         tail = newNode;
30         tail->next = head;
31     }
32     else {
33
34         newNode->next = head;
35         head = newNode;
36         tail->next = head;
37     }
38
39     size++;
40 }
41
42 void SingleLinkedList::add_back(int inputValue) {
43

```



```

44     Node* newNode = new Node;
45
46     newNode->data = inputValue;
47     newNode->next = nullptr;
48
49     if (is_empty()) {
50
51         head = newNode;
52         tail = newNode;
53
54         tail->next = head;
55     }
56     else {
57         tail->next = newNode;
58         tail = newNode;
59         tail->next = head;
60     }
61
62     size++;
63 }
64
65 void SingleLinkedList::add_idx(int inputValue,
66 int indexPosition) {
67     if (indexPosition < 0 || indexPosition >
68 size) {
69
70         cout << "Index tidak valid\n";
71         return;
72     }
73     if (indexPosition == 0) {
74         add_front(inputValue);
75     }
76 }

```

```

74         return;
75     }
76     if (indexPosition == size) {
77
78         add_back(inputValue);
79         return;
80     }
81     Node* newNode = new Node;
82
83     newNode->data = inputValue;
84     newNode->next = nullptr;
85
86     Node* currentNode = head;
87
88     for (int currentIndex = 0;
89         currentIndex < indexPosition - 1;
90         currentIndex++) {
91
92         currentNode = currentNode->next;
93     }
94
95     newNode->next = currentNode->next;
96
97     currentNode->next = newNode;
98
99     size++;
100 }
101
102 void SingleLinkedList::delete_front() {
103
104     if (is_empty()) {
105

```

```

106         cout << "Linked list kosong\n";
107         return;
108     }
109     if (head == tail) {
110
111         delete head;
112
113         head = nullptr;
114         tail = nullptr;
115     }
116     else {
117
118         Node* deletedNode = head;
119         head = head->next;
120         tail->next = head;
121
122         delete deletedNode;
123     }
124
125     size--;
126 }
127
128 void SingleLinkedList::delete_back() {
129     if (is_empty()) {
130
131         cout << "Linked list kosong\n";
132         return;
133     }
134     if (head == tail) {
135
136         delete head;
137

```

```

138         head = nullptr;
139         tail = nullptr;
140     }
141     else {
142
143         Node* currentNode = head;
144         while (currentNode->next != tail) {
145
146             currentNode = currentNode->next;
147         }
148         delete tail;
149         tail = currentNode;
150         tail->next = head;
151     }
152
153     size--;
154 }
155 void SingleLinkedList::delete_idx(int
indexPosition) {
156     if (indexPosition < 0 || indexPosition >=
size) {
157
158         cout << "Index tidak valid\n";
159         return;
160     }
161     if (indexPosition == 0) {
162
163         delete_front();
164         return;
165     }
166     if (indexPosition == size - 1) {
167

```

```

168         delete_back();
169         return;
170     }
171
172     Node* currentNode = head;
173     for (int currentIndex = 0;
174         currentIndex < indexPosition - 1;
175         currentIndex++) {
176
177         currentNode = currentNode->next;
178     }
179
180     Node* deletedNode = currentNode->next;
181     currentNode->next = deletedNode->next;
182
183     delete deletedNode;
184
185     size--;
186 }
187
188 void SingleLinkedList::display() {
189
190     if (is_empty()) {
191
192         cout << "Linked list kosong\n";
193         return;
194     }
195
196     Node* currentNode = head;
197
198     cout << "Isi Linked List : ";
199     do {

```

```

200         cout << currentNode->data << " ";
201
202         currentNode = currentNode->next;
203
204     } while (currentNode != head);
205
206     cout << endl;
207 }
208
209 int SingleLinkedList::get(int indexPosition) {
210     if (indexPosition < 0 || indexPosition >=
211 size) {
212
213         cout << "Index tidak valid\n";
214         return -1;
215     }
216
217     Node* currentNode = head;
218     for (int currentIndex = 0;
219         currentIndex < indexPosition;
220         currentIndex++) {
221
222         currentNode = currentNode->next;
223     }
224
225     return currentNode->data;
226 }
227
228 void SingleLinkedList::set(int inputValue,
229                             int indexPosition) {
230

```

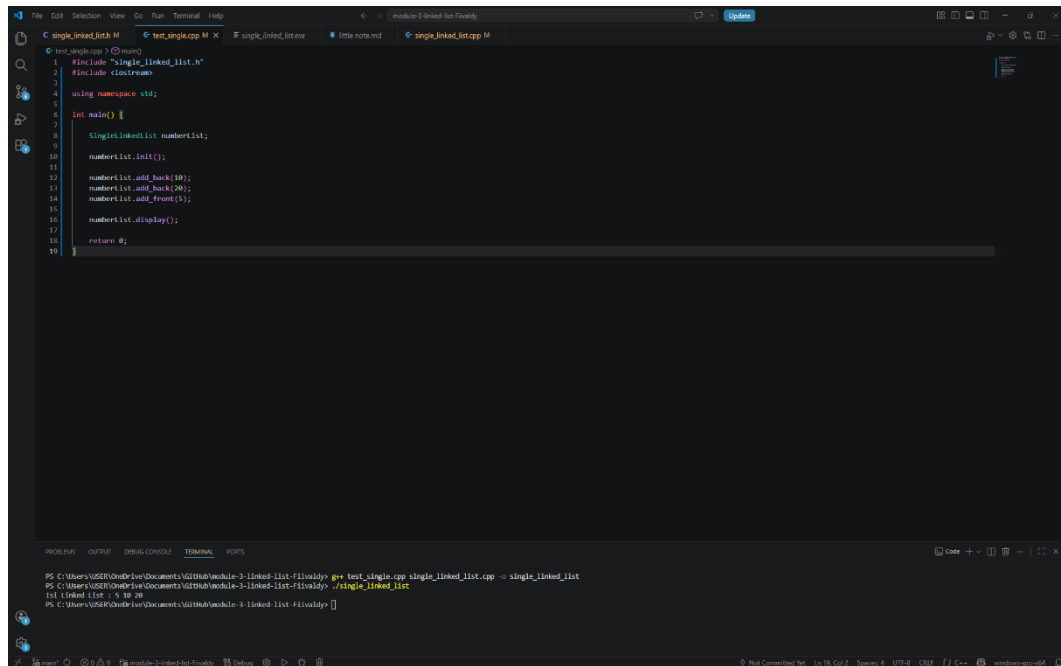
231	if (indexPosition < 0 indexPosition >=
232	size) {
233	cout << "Index tidak valid\n";
234	return;
235	}
236	
237	Node* currentNode = head;
238	for (int currentIndex = 0;
239	currentIndex < indexPosition;
240	currentIndex++) {
241	
242	currentNode = currentNode->next;
243	}
244	currentNode->data = inputValue;
245	}
246	void SingleLinkedList::clear() {
247	while (!is_empty()) {
248	
249	delete_front();
250	}
251	
252	head = nullptr;
253	tail = nullptr;
254	
255	size = 0;
256	}

Tabel 3 Source Code File test_single.cpp

1	#include "single_linked_list.h"
2	#include <iostream>
3	

4	using namespace std;
5	
6	int main() {
7	
8	SingleLinkedList numberList;
9	
10	numberList.init();
11	
12	numberList.add_back(10);
13	numberList.add_back(20);
14	numberList.add_front(5);
15	
16	numberList.display();
17	
18	return 0;
19	}

B. Output Program



```
1 #include "single_linked_list.h"
2 #include <iostream>
3
4 using namespace std;
5
6 int main() {
7     SinglyLinkedList numberList;
8
9     numberList.init();
10
11     numberList.add_back(10);
12     numberList.add_back(20);
13     numberList.add_front(5);
14
15     numberList.display();
16
17     return 0;
18 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\USER\OneDrive\Documents\GitHub\module-3-linked-list-filevaldy> g++ test_single.cpp single_linked_list.cpp -o single_linked_list
PS C:\Users\USER\OneDrive\Documents\GitHub\module-3-linked-list-filevaldy> ./single_linked_list
1st linked list : 5 10 20
PS C:\Users\USER\OneDrive\Documents\GitHub\module-3-linked-list-filevaldy>
```

Gambar 1 Screenshot Output Program Soal 1

C. Pembahasan

Fungsi `clear()` pada implementasi Circular Single Linked List dirancang untuk memastikan seluruh node yang telah dialokasikan di dalam heap dapat dihapus dengan benar sehingga tidak terjadi *memory leak*. Pada implementasi program, proses penghapusan dilakukan menggunakan perulangan `while (!is_empty())` yang akan terus memanggil fungsi `delete_front()` hingga seluruh node habis terhapus. Dengan cara ini setiap node yang sebelumnya dibuat menggunakan sintaks `new Node` akan dibebaskan kembali menggunakan `delete`, sehingga memori yang dipakai program dapat dikembalikan ke sistem operasi. Setelah seluruh node terhapus, pointer `head` dan `tail` dikembalikan menjadi `nullptr`, sedangkan `size` diubah menjadi 0. Alur ini penting karena tanpa proses penghapusan manual, node yang sudah tidak digunakan masih tetap berada di heap dan menyebabkan kebocoran memori.

Seluruh manajemen memori pada program dilakukan secara manual menggunakan alokasi heap. Hal ini terlihat pada sintaks `Node* newNode = new Node;` yang digunakan ketika membuat node baru. Penggunaan heap dipilih karena jumlah node pada linked list bersifat dinamis dan dapat bertambah maupun berkurang selama program berjalan. Dengan alokasi heap, ukuran linked list tidak perlu ditentukan sejak awal seperti array statis. Namun, karena memori dialokasikan secara manual maka setiap node yang sudah tidak digunakan wajib dihapus menggunakan `delete`. Oleh sebab itu fungsi-fungsi seperti `delete_front()`, `delete_back()`, `delete_idx()`, dan `clear()` menjadi sangat penting dalam menjaga efisiensi penggunaan memori.

Pada fungsi `init()`, kompleksitas waktu yang digunakan adalah $O(1)$ karena program hanya mengubah nilai beberapa pointer dan variabel tanpa melakukan perulangan. Kompleksitas ruangnya juga $O(1)$ karena tidak ada penggunaan memori tambahan selain variabel yang sudah ada. Fungsi `is_empty()` juga memiliki kompleksitas waktu $O(1)$ karena hanya melakukan satu kali pengecekan kondisi pada pointer `head`. Kompleksitas ruangnya tetap $O(1)$. Pada fungsi `add_front()` dan `add_back()`, kompleksitas waktunya adalah $O(1)$ karena penambahan node dilakukan langsung pada bagian depan atau belakang tanpa perlu menelusuri seluruh linked list. Kompleksitas ruangnya adalah $O(1)$ karena hanya

membutuhkan satu node baru sebagai memori tambahan. Fungsi `add_idx()` memiliki kompleksitas waktu $O(n)$ karena program harus melakukan traversal atau penelusuran node hingga mencapai posisi index tertentu. Pada kondisi terburuk, traversal dilakukan sampai mendekati node terakhir. Kompleksitas ruangnya tetap $O(1)$ karena hanya menggunakan satu node tambahan. Fungsi `delete_front()` mempunyai kompleksitas waktu $O(1)$ karena node pertama dapat langsung dihapus tanpa traversal. Kompleksitas ruangnya juga $O(1)$. Pada fungsi `delete_back()`, kompleksitas waktunya adalah $O(n)$ karena program harus mencari node sebelum tail menggunakan traversal. Kompleksitas ruangnya tetap $O(1)$ karena tidak membutuhkan struktur data tambahan. Fungsi `delete_idx()` juga memiliki kompleksitas waktu $O(n)$ karena program perlu bergerak menuju node sebelum posisi target. Kompleksitas ruangnya tetap $O(1)$. Pada fungsi `display()`, kompleksitas waktunya adalah $O(n)$ karena seluruh node harus ditampilkan satu per satu hingga kembali ke head. Kompleksitas ruangnya adalah $O(1)$ karena tidak menggunakan memori tambahan yang signifikan. Fungsi `get()` dan `set()` memiliki kompleksitas waktu $O(n)$ karena keduanya membutuhkan traversal menuju index tertentu sebelum data diambil atau diubah. Kompleksitas ruang kedua fungsi tersebut tetap $O(1)$. Sementara itu fungsi `clear()` memiliki kompleksitas waktu $O(n)$ karena setiap node harus dihapus satu per satu hingga linked list kosong. Kompleksitas ruangnya tetap $O(1)$ karena proses penghapusan dilakukan langsung tanpa memerlukan struktur tambahan.

Seluruh operasi linked list mempertahankan hubungan circular antara tail dan head. Setiap kali node ditambahkan atau dihapus, pointer `tail->next` selalu diperbarui agar tetap menunjuk ke head. Dengan demikian struktur circular linked list tetap terjaga pada seluruh kondisi, baik ketika linked list kosong, memiliki satu node, maupun banyak node. Selain itu validasi index pada fungsi seperti `add_idx()`, `delete_idx()`, `get()`, dan `set()` memastikan program tidak mengakses memori di luar batas linked list. Penggunaan delete pada setiap proses penghapusan juga menjamin memori yang sudah tidak digunakan dapat dibebaskan kembali sehingga program menjadi lebih aman dan efisien.

Kode file **single_linked_list.cpp** merupakan implementasi dari struktur data *Circular Single Linked List*, yaitu linked list yang node terakhirnya kembali menunjuk ke node pertama sehingga membentuk lingkaran. Pada bagian awal program, file header dipanggil menggunakan sintaks `#include "single_linked_list.h"` pada baris [1] agar seluruh deklarasi struktur dan fungsi yang telah dibuat sebelumnya dapat digunakan kembali pada file implementasi ini. Selain itu, library `iostream` pada baris [2] digunakan untuk proses input dan output seperti menampilkan data linked list ke layar.

Fungsi `init()` pada baris [6] digunakan untuk menginisialisasi linked list agar berada dalam kondisi kosong. Pada bagian ini variabel `head` dan `tail` diubah menjadi `nullptr` pada baris [8] dan [9], yang menandakan belum terdapat node di dalam linked list. Sementara itu variabel `size` pada baris [11] diatur menjadi 0 untuk menunjukkan jumlah data awal masih kosong. Fungsi `is_empty()` pada baris [14] berfungsi untuk memeriksa apakah linked list masih kosong atau tidak. Pemeriksaan dilakukan menggunakan sintaks `return head == nullptr;` pada baris [16]. Jika `head` bernilai `nullptr`, maka fungsi akan menghasilkan nilai `true`, yang berarti belum terdapat node di dalam linked list. Pada fungsi `add_front()` di baris [19], program membuat node baru menggunakan sintaks `Node* newNode = new Node;` pada baris [21]. Sintaks ini berfungsi untuk memesan memori baru secara dinamis. Kemudian data yang dimasukkan pengguna disimpan ke dalam node melalui `newNode->data = inputValue;` pada baris [23]. Jika linked list masih kosong, maka node baru akan menjadi `head` dan `tail` sekaligus seperti terlihat pada baris [28] dan [29]. Selanjutnya sintaks `tail->next = head;` pada baris [30] menjadi bagian penting karena membuat node terakhir kembali menunjuk ke node pertama sehingga struktur linked list menjadi circular.

Fungsi `add_back()` pada baris [40] memiliki tujuan untuk menambahkan data di bagian belakang linked list. Ketika linked list tidak kosong, program menghubungkan node terakhir lama ke node baru menggunakan sintaks `tail->next = newNode;` pada baris [54]. Setelah itu posisi `tail` dipindahkan ke node baru pada

baris [55]. Bagian penting lainnya adalah `tail->next = head`; pada baris [56] yang memastikan hubungan circular tetap terjaga setelah penambahan data.

Pada fungsi `add_idx()` di baris [63], program memungkinkan penambahan data berdasarkan posisi index tertentu. Validasi index dilakukan terlebih dahulu pada baris [64] agar pengguna tidak memasukkan posisi yang melebihi ukuran linked list. Proses pencarian posisi node dilakukan menggunakan perulangan `for` pada baris [84] hingga [86]. Perulangan ini berjalan sampai node sebelum posisi tujuan ditemukan. Setelah itu node baru disisipkan menggunakan sintaks `newNode->next = currentNode->next`; pada baris [91] dan `currentNode->next = newNode`; pada baris [93]. Fungsi `delete_front()` pada baris [99] digunakan untuk menghapus node pertama. Jika linked list hanya memiliki satu node, maka memori node dihapus menggunakan sintaks `delete head`; pada baris [109]. Namun jika jumlah node lebih dari satu, maka head dipindahkan ke node berikutnya menggunakan `head = head->next`; pada baris [118]. Kemudian hubungan circular diperbarui melalui `tail->next = head`; pada baris [119]. Pada fungsi `delete_back()` di baris [127], program mencari node sebelum tail menggunakan perulangan `while` pada baris [142]. Setelah ditemukan, node terakhir dihapus menggunakan `delete tail`; pada baris [147]. Kemudian tail dipindahkan ke node sebelumnya pada baris [148], dan hubungan circular diperbaiki kembali melalui `tail->next = head`; pada baris [149]. Fungsi `delete_idx()` pada baris [155] digunakan untuk menghapus node berdasarkan posisi index tertentu. Program terlebih dahulu melakukan validasi index pada baris [156]. Selanjutnya pencarian node sebelum target dilakukan melalui perulangan `for` pada baris [174] hingga [176]. Node target kemudian dilewati menggunakan sintaks `currentNode->next = deletedNode->next`; pada baris [182], sehingga node tersebut tidak lagi terhubung dengan linked list sebelum akhirnya dihapus menggunakan `delete deletedNode`; pada baris [184]. Fungsi `display()` pada baris [190] digunakan untuk menampilkan seluruh isi linked list. Karena struktur yang digunakan adalah circular linked list, maka program memakai perulangan `do while` pada baris [201] hingga [207]. Perulangan ini akan terus berjalan sampai pointer kembali ke head, sehingga seluruh node dapat ditampilkan tanpa terjadi perulangan tak terbatas. Pada fungsi `get()` di baris [213], program mengambil data berdasarkan index tertentu. Proses perpindahan node dilakukan melalui perulangan `for` pada baris

[222] hingga [224]. Setelah posisi node ditemukan, data dikembalikan menggunakan sintaks `return currentNode->data;` pada baris [229].

Fungsi `set()` pada baris [232] digunakan untuk mengubah isi data pada index tertentu. Setelah posisi node ditemukan melalui proses perulangan pada baris [242] hingga [244], nilai data lama diganti menggunakan sintaks `currentNode->data = inputValue;` pada baris [247]. Fungsi `clear()` pada baris [249] digunakan untuk menghapus seluruh isi linked list. Penghapusan dilakukan secara berulang menggunakan `while (!is_empty())` pada baris [250], yang akan terus memanggil fungsi `delete_front()` sampai seluruh node habis terhapus. Setelah selesai, `head`, `tail`, dan `size` dikembalikan lagi ke kondisi awal kosong pada baris [254] hingga [257].

Pada Kode header file **single_linked_list.h** digunakan sebagai tempat deklarasi struktur data dan fungsi-fungsi yang akan dipakai pada program Circular Single Linked List. Pada bagian awal terdapat sintaks `#ifndef SINGLE_LINKED_LIST` di baris [1] dan `#define SINGLE_LINKED_LIST` pada baris [2]. Kedua sintaks ini berfungsi sebagai *header guard*, yaitu untuk mencegah file header dipanggil berulang kali saat proses kompilasi. Tanpa bagian ini, program dapat mengalami error karena deklarasi struktur atau fungsi dianggap dibuat lebih dari satu kali. Struktur `Node` pada baris [4] digunakan sebagai komponen utama dalam linked list. Di dalamnya terdapat variabel `data` pada baris [5] yang berfungsi menyimpan nilai bertipe integer. Selain itu terdapat pointer `next` pada baris [6] yang digunakan untuk menyimpan alamat node berikutnya. Pointer inilah yang membuat setiap node dapat saling terhubung membentuk linked list.

Struktur `SingleLinkedList` pada baris [9] digunakan sebagai wadah utama untuk mengatur seluruh operasi linked list. Variabel `head` pada baris [11] berfungsi menunjuk node pertama, sedangkan `tail` menunjuk node terakhir. Nilai awal keduanya diatur menjadi `nullptr` untuk menandakan linked list masih kosong. Pada baris [13], terdapat variabel `size` yang digunakan untuk menyimpan jumlah node yang ada di dalam linked list. Deklarasi fungsi `init()` pada baris [15] digunakan untuk melakukan inisialisasi awal linked list. Fungsi `is_empty()` pada baris [16] berfungsi

mengecek apakah linked list masih kosong atau sudah memiliki data. Kedua fungsi ini menjadi dasar penting sebelum melakukan operasi lain pada linked list.

Pada baris [18] hingga [20], terdapat fungsi `add_front()`, `add_back()`, dan `add_idx()`. Ketiga fungsi ini digunakan untuk menambahkan data ke dalam linked list. Fungsi `add_front()` menambahkan node di bagian depan, `add_back()` menambahkan node di bagian belakang, sedangkan `add_idx()` digunakan untuk menyisipkan node pada posisi index tertentu. Selanjutnya, pada baris [22] hingga [24], terdapat fungsi `delete_front()`, `delete_back()`, dan `delete_idx()`. Fungsi-fungsi tersebut digunakan untuk menghapus node dari linked list. Penghapusan dapat dilakukan dari bagian depan, belakang, maupun berdasarkan posisi index tertentu. Fungsi `display()` pada baris [26] digunakan untuk menampilkan seluruh isi linked list ke layar. Fungsi ini penting untuk memastikan data berhasil disimpan atau dihapus sesuai proses yang dilakukan pengguna.

Pada baris [28], fungsi `get()` digunakan untuk mengambil nilai data berdasarkan index tertentu. Sedangkan fungsi `set()` pada baris [30] digunakan untuk mengubah nilai data pada posisi tertentu di dalam linked list. Fungsi `clear()` pada baris [32] digunakan untuk menghapus seluruh node yang ada di linked list. Fungsi ini sangat penting karena membantu mencegah terjadinya *memory leak*, yaitu kondisi ketika memori yang sudah tidak digunakan masih tetap tersimpan dan tidak dibebaskan dari program.

Kode pada file **single_test.cpp** berfungsi sebagai program utama untuk menjalankan dan menguji implementasi Circular Single Linked List yang telah dibuat sebelumnya. Pada baris [1], program memanggil file header menggunakan sintaks `#include "single_linked_list.h"` agar seluruh struktur data dan fungsi yang telah dideklarasikan pada file header dapat digunakan di dalam program

utama. Kemudian pada baris [2], library `iostream` digunakan untuk mendukung proses input dan output seperti menampilkan data ke layar.

Sintaks `using namespace std;` pada baris [4] digunakan agar penulisan fungsi standar C++ seperti `cout` dan `endl` tidak perlu diawali dengan `std::`. Hal ini membuat penulisan kode menjadi lebih sederhana dan mudah dibaca.

Fungsi utama program dimulai pada baris [6] melalui `int main()`. Fungsi `main()` merupakan titik awal eksekusi program C++. Seluruh perintah yang berada di dalam fungsi ini akan dijalankan pertama kali ketika program dieksekusi.

Pada baris [8], program membuat sebuah objek bernama `numberList` dari struktur `SingleLinkedList` menggunakan sintaks `SingleLinkedList numberList;`. Objek ini nantinya digunakan untuk mengakses seluruh fungsi linked list seperti menambah, menghapus, dan menampilkan data.

Selanjutnya pada baris [10], fungsi `numberList.init();` dipanggil untuk melakukan inisialisasi awal linked list. Fungsi ini mengatur `head` dan `tail` menjadi kosong (`nullptr`) serta mengatur ukuran linked list menjadi 0. Proses ini penting agar linked list berada dalam kondisi siap digunakan sebelum dilakukan operasi lainnya.

Pada baris [12] dan [13], program menambahkan data ke bagian belakang linked list menggunakan fungsi `add_back()`. Nilai 10 dimasukkan terlebih dahulu, kemudian diikuti nilai 20. Setelah itu pada baris [14], fungsi `add_front(5)` digunakan untuk menambahkan data 5 di bagian depan linked list. Dengan proses tersebut, urutan data di dalam linked list menjadi 5 -> 10 -> 20. Fungsi `numberList.display();` pada baris [16] digunakan untuk menampilkan seluruh isi linked list ke layar. Fungsi ini akan membaca node satu per satu mulai dari `head` hingga kembali lagi ke `head` karena struktur yang digunakan adalah circular linked list.

SOAL 2

Si Palui dan Relikui Siklus Resonansi

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh N buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh K langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ($K = K + 1$).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ($K = K - 1$).
- Jika pada suatu titik energi lompatan habis atau negatif ($K \leq 0$), sistem akan mengalami **reset** dan mengembalikan nilai lompatan ke nilai K awal (saat pancaran energi pertama kali diinisialisasi).

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

Batasan

- $1 \leq N \leq 10^4$
- $1 \leq K \leq 10^9$
- $1 \leq A_i \leq 10^6$ (Nilai ukiran pada batu)
- Program wajib menggunakan implementasi struktur data yang dibuat dari Soal 1

Format Input

N K
A1 A2 ... AN

Format Output

Sebuah bilangan bulat tunggal yang merupakan nilai ukiran pada batu terakhir yang tersisa.

Contoh Kasus

Input	Output
5 2 1 4 3 6 2	6
4 5 10 11 12 13	12

Penjelasan Contoh Kasus

Pada contoh pertama, susunan batu awal adalah $[1,4,3,6,2]$ dengan $K = 2$. Dari batu pertama, lompatan 2 langkah mendarat di batu 4 sehingga batu tersebut dihancurkan. Karena 4 genap, nilai K bertambah menjadi 3 dan susunan berubah menjadi $[1,3,6,2]$.

Selanjutnya, dengan $K = 3$, lompatan mendarat di batu 2. Batu 2 dihancurkan dan karena genap, K kembali bertambah menjadi 4. Susunan kini menjadi $[1,3,6]$.

Pada fase berikutnya dengan $K = 4$, lompatan mendarat di batu 1. Karena 1 ganjil, setelah dihancurkan nilai K berkurang menjadi 3, menyisakan $[3,6]$.

Terakhir, dengan $K = 3$, lompatan kembali mendarat di batu 3 sehingga batu tersebut dihancurkan. Struktur kini hanya menyisakan batu bernilai 6, sehingga keluaran akhirnya adalah 6.

A. Source Code

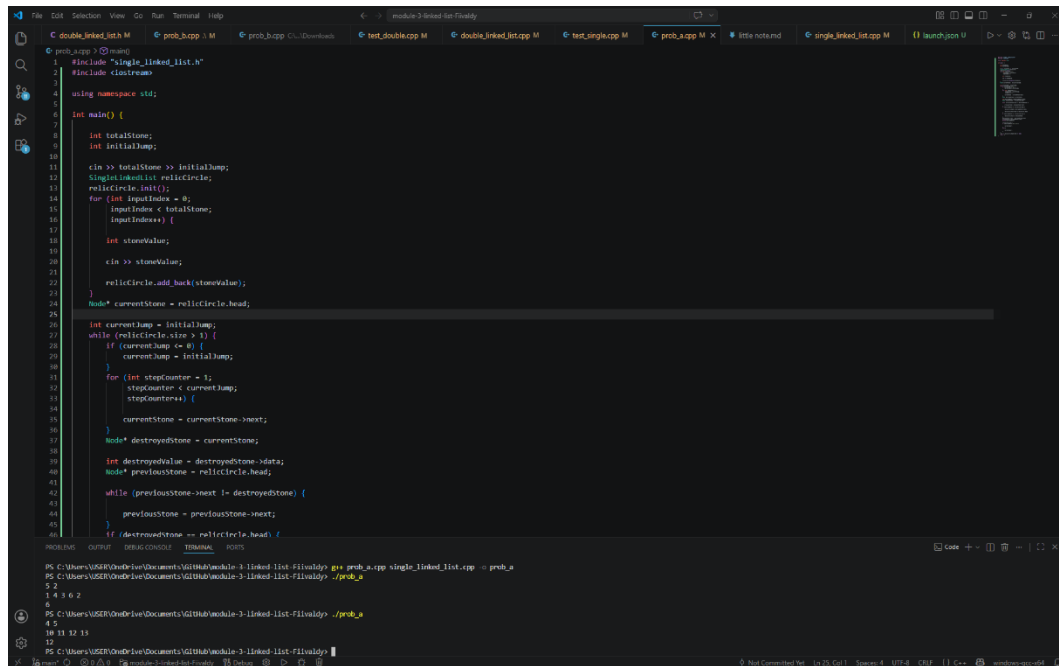
Tabel 4 Source Code File prob_a.cpp

1	<code>#include "single_linked_list.h"</code>
2	<code>#include <iostream></code>
3	
4	<code>using namespace std;</code>
5	
6	<code>int main() {</code>
7	
8	<code> int totalStone;</code>
9	<code> int initialJump;</code>
10	
11	<code> cin >> totalStone >> initialJump;</code>
12	<code> SingleLinkedList relicCircle;</code>
13	<code> relicCircle.init();</code>
14	<code> for (int inputIndex = 0;</code>
15	<code> inputIndex < totalStone;</code>
16	<code> inputIndex++) {</code>
17	
18	<code> int stoneValue;</code>
19	
20	<code> cin >> stoneValue;</code>
21	
22	<code> relicCircle.add_back(stoneValue);</code>
23	<code> }</code>
24	<code> Node* currentStone = relicCircle.head;</code>
25	
26	<code> int currentJump = initialJump;</code>
27	<code> while (relicCircle.size > 1) {</code>
28	<code> if (currentJump <= 0) {</code>
29	<code> currentJump = initialJump;</code>
30	<code> }</code>

31	for (int stepCounter = 1;
32	stepCounter < currentJump;
33	stepCounter++) {
34	
35	currentStone = currentStone->next;
36	}
37	Node* destroyedStone = currentStone;
38	
39	int destroyedValue = destroyedStone->
40	data;
41	Node* previousStone = relicCircle.head;
42	while (previousStone->next !=
43	destroyedStone) {
44	previousStone = previousStone->next;
45	}
46	if (destroyedStone == relicCircle.head) {
47	relicCircle.head = destroyedStone->
48	next;
49	relicCircle.tail->next =
50	relicCircle.head;
51	}
52	if (destroyedStone == relicCircle.tail) {
53	relicCircle.tail = previousStone;
54	}
55	previousStone->next = destroyedStone->
56	next;
57	currentStone = destroyedStone->next;

58	delete destroyedStone;
59	
60	relicCircle.size--;
61	if (destroyedValue % 2 == 0) {
62	
63	currentJump++;
64	}
65	else {
66	
67	currentJump--;
68	}
69	}
70	cout << relicCircle.head->data << endl;
71	return 0;
72	}

B. Output Program



```
1 #include "single_linked_list.h"
2 #include <iostream>
3
4 using namespace std;
5
6 int main() {
7     int totalStone;
8     int initialJump;
9
10    cin >> totalStone >> initialJump;
11    SingleLinkedList relicCircle;
12    relicCircle.init();
13    for (int inputIndex = 0;
14         inputIndex < totalStone;
15         inputIndex++) {
16        int stoneValue;
17        cin >> stoneValue;
18        relicCircle.add_back(stoneValue);
19    }
20    Node* currentStone = relicCircle.head;
21
22    int currentJump = initialJump;
23    while (relicCircle.size > 1) {
24        if (currentJump <= 0) {
25            currentJump = initialJump;
26        }
27        for (int stepCounter = 1;
28             stepCounter < currentJump;
29             stepCounter++) {
30            currentStone = currentStone->next;
31        }
32        Node* destroyedStone = currentStone;
33        int destroyedValue = destroyedStone->data;
34        Node* previousStone = relicCircle.head;
35        while (previousStone->next != destroyedStone) {
36            previousStone = previousStone->next;
37        }
38        if (destroyedStone == relicCircle.head) {
39            relicCircle.head = previousStone->next;
40        }
41        previousStone->next = previousStone->next->next;
42        delete destroyedStone;
43    }
44    cout << "The remaining stone is: " << relicCircle.head->data << endl;
45    return 0;
46 }
```

Output: 5 2
1 4 3 6 2
5
4 5
18 11 12 13
12

Gambar 2 Screenshot Output Program Soal 2

C. Pembahasan

Secara alur, program bekerja dengan cara menyusun seluruh batu ke dalam circular linked list, kemudian melakukan perpindahan sesuai nilai lompatan yang terus berubah berdasarkan aturan genap dan ganjil. Setiap kali pointer berhenti pada suatu batu, batu tersebut dihancurkan dan dihapus dari linked list. Proses ini berlangsung berulang sampai hanya tersisa satu batu terakhir sebagai hasil akhir program.

Kode pada file `prob_a.cpp` merupakan program simulasi penghancuran batu relik menggunakan struktur data Circular Single Linked List. Pada baris [1], file `single_linked_list.h` dipanggil agar seluruh fungsi dan struktur linked list yang telah dibuat sebelumnya dapat digunakan kembali dalam program. Kemudian library `iostream` pada baris [2] digunakan untuk proses input dan output data.

Fungsi utama program dimulai pada baris [6] melalui `int main()`. Pada bagian ini program pertama kali membaca jumlah batu dan nilai awal lompatan menggunakan `cin >> totalStone >> initialJump`; pada baris [11]. Variabel `totalStone` menyimpan jumlah batu relik, sedangkan `initialJump` digunakan sebagai nilai awal banyaknya langkah perpindahan. Pada baris [12], program membuat objek `relicCircle` dari struktur `SingleLinkedList`. Objek ini digunakan untuk menyimpan seluruh batu relik dalam bentuk circular linked list. Setelah itu fungsi `relicCircle.init()`; pada baris [13] dipanggil untuk memastikan linked list berada dalam kondisi kosong sebelum data dimasukkan. Proses input batu dilakukan menggunakan perulangan `for` pada baris [14] hingga [23]. Pada setiap perulangan, program membaca nilai batu menggunakan `cin >> stoneValue`; pada baris [20]. Nilai tersebut kemudian dimasukkan ke bagian belakang linked list menggunakan fungsi `relicCircle.add_back(stoneValue)`; pada baris [22]. Dengan cara ini seluruh batu tersusun membentuk lingkaran sesuai aturan soal. Pada baris [24], pointer `currentStone` diatur menuju `relicCircle.head`. Pointer ini digunakan untuk menandai posisi batu yang sedang aktif pada proses perpindahan. Kemudian nilai lompatan saat ini disimpan pada variabel `currentJump` pada baris [26].

Perulangan utama program berada pada baris [27] menggunakan kondisi `while (relicCircle.size > 1)`. Perulangan ini akan terus berjalan selama jumlah batu masih lebih dari satu. Artinya proses penghancuran batu dilakukan terus menerus sampai hanya tersisa satu batu terakhir. Pada baris [28] hingga [30], program memeriksa apakah nilai lompatan sudah habis atau bernilai kurang dari sama dengan nol. Jika kondisi tersebut terjadi, maka nilai lompatan dikembalikan lagi ke nilai awal menggunakan `currentJump = initialJump`; Hal ini sesuai aturan pada soal bahwa energi akan kembali ke nilai awal ketika sudah tidak dapat digunakan.

Proses perpindahan batu dilakukan pada baris [31] hingga [35] menggunakan perulangan `for`. Pointer `currentStone` dipindahkan maju menggunakan sintaks `currentStone = currentStone->next`; pada baris [35]. Perulangan dimulai dari angka 1 karena posisi awal pointer sudah dianggap sebagai langkah pertama. Setelah perpindahan selesai, batu tempat pointer berhenti akan menjadi target penghancuran. Pada baris [37], node target disimpan ke dalam pointer `destroyedStone`. Nilai batu tersebut juga disimpan pada variabel `destroyedValue` di baris [39] karena nantinya digunakan untuk menentukan perubahan nilai lompatan. Selanjutnya program mencari node sebelum target menggunakan perulangan `while` pada baris [42] hingga [45]. Langkah ini diperlukan agar hubungan antar node tetap tersambung setelah node target dihapus.

Jika batu yang dihancurkan merupakan head, maka posisi head dipindahkan ke node berikutnya menggunakan `relicCircle.head = destroyedStone->next`; pada baris [48]. Setelah itu hubungan circular diperbaiki melalui `relicCircle.tail->next = relicCircle.head`; pada baris [50]. Sementara itu jika batu target merupakan tail, maka posisi tail dipindahkan ke node sebelumnya menggunakan `relicCircle.tail = previousStone`; pada baris [54]. Pada baris [56], hubungan linked list diperbaiki menggunakan `previousStone->next = destroyedStone->next`; agar node yang dihancurkan tidak lagi terhubung dengan list. Kemudian pointer dipindahkan ke node berikutnya menggunakan `currentStone = destroyedStone->next`; pada baris [57]. Node target lalu dihapus dari memori menggunakan `delete destroyedStone`; pada baris [58] agar tidak terjadi kebocoran memori. Jumlah node dalam linked list dikurangi pada baris [60] menggunakan `relicCircle.size--`; Setelah penghancuran selesai, program memeriksa apakah nilai batu yang dihancurkan genap atau ganjil

pada baris [61]. Jika genap maka nilai lompatan ditambah satu menggunakan `currentJump++`; pada baris [63]. Namun jika ganjil maka nilai lompatan dikurangi satu menggunakan `currentJump--`; pada baris [67]. Ketika hanya tersisa satu batu, program menampilkan nilainya menggunakan `cout << relicCircle.head->data << endl`; pada baris [70]. Nilai tersebut merupakan batu terakhir yang bertahan setelah seluruh proses penghancuran selesai dilakukan.

SOAL 3

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList`. beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`. Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap. Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 5 Source Code File `double_linked_list.h`

1	<code>#ifndef DOUBLE_LINKED_LIST</code>
2	<code>#define DOUBLE_LINKED_LIST</code>
3	
4	<code>struct Node {</code>
5	<code> char data;</code>
6	<code> Node* next;</code>
7	<code> Node* prev;</code>
8	<code>};</code>
9	
10	<code>struct DoubleLinkedList {</code>
11	
12	<code> Node* head = nullptr;</code>
13	<code> Node* tail = nullptr;</code>
14	
15	<code> int size = 0;</code>
16	
17	<code> void init();</code>
18	<code> bool is_empty();</code>
19	
20	<code> void add_front(char inputValue);</code>

21	void add_back(char inputValue);
22	void add_idx(char inputValue, int indexPosition);
23	
24	void delete_front();
25	void delete_back();
26	void delete_idx(int indexPosition);
27	
28	void display();
29	
30	char get(int indexPosition);
31	
32	void set(char inputValue, int indexPosition);
33	
34	void clear();
35	};
36	
37	#endif

Tabel 6 Source Code File double_linked_list.h

1	#include "double_linked_list.h"
2	#include <iostream>
3	
4	using namespace std;
5	void DoubleLinkedList::init() {
6	
7	head = nullptr;
8	tail = nullptr;
9	
10	size = 0;
11	}

12	bool DoubleLinkedList::is_empty() {
13	
14	return head == nullptr;
15	}
16	
	void DoubleLinkedList::add_front(char inputValue)
17	{
18	Node* newNode = new Node;
19	newNode->data = inputValue;
20	if (is_empty()) {
21	
22	newNode->next = newNode;
23	newNode->prev = newNode;
24	
25	head = newNode;
26	tail = newNode;
27	}
28	else {
29	newNode->next = head;
30	newNode->prev = tail;
31	head->prev = newNode;
32	tail->next = newNode;
33	head = newNode;
34	}
35	
36	size++;
37	}
38	
	void DoubleLinkedList::add_back(char inputValue)
39	{
40	
41	Node* newNode = new Node;

```

42     newNode->data = inputValue;
43     if (is_empty()) {
44
45         newNode->next = newNode;
46         newNode->prev = newNode;
47
48         head = newNode;
49         tail = newNode;
50     }
51     else {
52         newNode->next = head;
53         newNode->prev = tail;
54         tail->next = newNode;
55         head->prev = newNode;
56         tail = newNode;
57     }
58
59     size++;
60 }
61
62 void DoubleLinkedList::add_idx(char inputValue,
63                                int indexPosition)
64 {
65     if (indexPosition < 0 ||
66         indexPosition > size) {
67
68         cout << "Index tidak valid\n";
69         return;
70     }
71     if (indexPosition == 0) {
72

```

73	add_front(inputValue);
74	return;
75	}
76	
77	if (indexPosition == size) {
78	
79	add_back(inputValue);
80	return;
81	}
82	
83	Node* currentNode = head;
84	for (int currentIndex = 0;
85	currentIndex < indexPosition;
86	currentIndex++) {
87	
88	currentNode = currentNode->next;
89	}
90	
91	Node* newNode = new Node;
92	
93	newNode->data = inputValue;
94	newNode->next = currentNode;
95	newNode->prev = currentNode->prev;
96	
97	currentNode->prev->next = newNode;
98	currentNode->prev = newNode;
99	
100	size++;
101	}
102	void DoubleLinkedList::delete_front() {
103	if (is_empty()) {
104	

```

105         cout << "Linked list kosong\n";
106         return;
107     }
108
109     if (head == tail) {
110
111         delete head;
112
113         head = nullptr;
114         tail = nullptr;
115     }
116     else {
117
118         Node* deletedNode = head;
119         head = head->next;
120         head->prev = tail;
121         tail->next = head;
122
123         delete deletedNode;
124     }
125
126     size--;
127 }
128 void DoubleLinkedList::delete_back() {
129     if (is_empty()) {
130
131         cout << "Linked list kosong\n";
132         return;
133     }
134     if (head == tail) {
135
136         delete tail;

```

```

137         head = nullptr;
138         tail = nullptr;
139     }
140     else {
141         Node* deletedNode = tail;
142         tail = tail->prev;
143         tail->next = head;
144         head->prev = tail;
145
146         delete deletedNode;
147     }
148
149     size--;
150 }
151 void DoubleLinkedList::delete_idx(int
152 indexPosition) {
153     if (indexPosition < 0 ||
154         indexPosition >= size) {
155
156         cout << "Index tidak valid\n";
157         return;
158     }
159     if (indexPosition == 0) {
160
161         delete_front();
162         return;
163     }
164     if (indexPosition == size - 1) {
165
166         delete_back();
167         return;

```



```

168     }
169
170     Node* currentNode = head;
171     for (int currentIndex = 0;
172         currentIndex < indexPosition;
173         currentIndex++) {
174
175         currentNode = currentNode->next;
176     }
177     currentNode->prev->next = currentNode->next;
178     currentNode->next->prev = currentNode->prev;
179
180     delete currentNode;
181
182     size--;
183 }
184 void DoubleLinkedList::display() {
185     if (is_empty()) {
186
187         cout << "Linked list kosong\n";
188         return;
189     }
190
191     Node* currentNode = head;
192
193     cout << "Isi Linked List : ";
194     do {
195
196         cout << currentNode->data << " ";
197
198         currentNode = currentNode->next;
199

```

```

200     } while (currentNode != head);
201
202     cout << endl;
203 }
204 char DoubleLinkedList::get(int indexPosition) {
205     if (indexPosition < 0 ||
206         indexPosition >= size) {
207
208         cout << "Index tidak valid\n";
209         return '\0';
210     }
211
212     Node* currentNode = head;
213     for (int currentIndex = 0;
214         currentIndex < indexPosition;
215         currentIndex++) {
216
217         currentNode = currentNode->next;
218     }
219
220     return currentNode->data;
221 }
222 void DoubleLinkedList::set(char inputValue,
223                             int indexPosition) {
224
225     if (indexPosition < 0 ||
226         indexPosition >= size) {
227
228         cout << "Index tidak valid\n";
229         return;
230     }
231

```

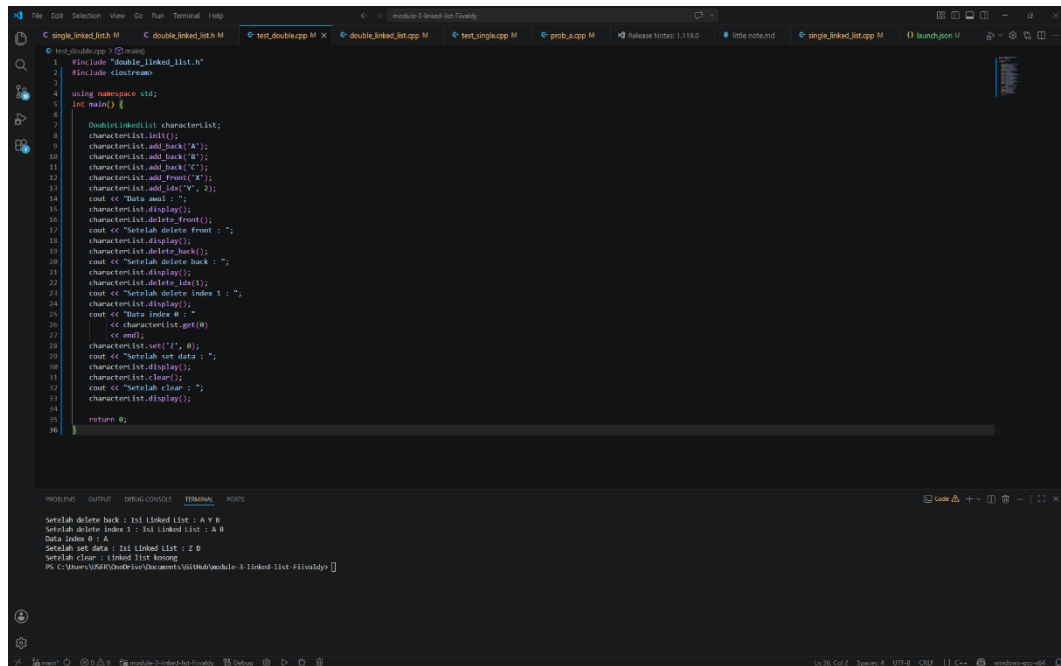
232	Node* currentNode = head;
233	for (int currentIndex = 0;
234	currentIndex < indexPosition;
235	currentIndex++) {
236	
237	currentNode = currentNode->next;
238	}
239	
240	currentNode->data = inputValue;
241	}
242	
243	void DoubleLinkedList::clear() {
244	while (!is_empty()) {
245	
246	delete_front();
247	}
248	
249	head = nullptr;
250	tail = nullptr;
251	
252	size = 0;
253	}

Tabel 7 Source Code File test_double.cpp

1	#include "double_linked_list.h"
2	#include <iostream>
3	
4	using namespace std;
5	int main() {
6	
7	DoubleLinkedList characterList;
8	characterList.init();

9	characterList.add_back('A');
10	characterList.add_back('B');
11	characterList.add_back('C');
12	characterList.add_front('X');
13	characterList.add_idx('Y', 2);
14	cout << "Data awal : ";
15	characterList.display();
16	characterList.delete_front();
17	cout << "Setelah delete front : ";
18	characterList.display();
19	characterList.delete_back();
20	cout << "Setelah delete back : ";
21	characterList.display();
22	characterList.delete_idx(1);
23	cout << "Setelah delete index 1 : ";
24	characterList.display();
25	cout << "Data index 0 : "
26	<< characterList.get(0)
27	<< endl;
28	characterList.set('Z', 0);
29	cout << "Setelah set data : ";
30	characterList.display();
31	characterList.clear();
32	cout << "Setelah clear : ";
33	characterList.display();
34	
35	return 0;
36	}

B. Output Program



```
1 #include "double_linked_list.h"
2 #include <iostream>
3
4 using namespace std;
5 int main() {
6
7     DoublyLinkedList characterList;
8     characterList.init();
9     characterList.add_back("A");
10    characterList.add_back("B");
11    characterList.add_back("C");
12    characterList.add_front("D");
13    characterList.add_id("V", 2);
14    cout << "Data awal : ";
15    characterList.display();
16    characterList.delete_front();
17    cout << "Setelah delete front : ";
18    characterList.display();
19    characterList.delete_back();
20    cout << "Setelah delete back : ";
21    characterList.display();
22    characterList.delete_id(1);
23    cout << "Setelah delete index 1 : ";
24    characterList.display();
25    cout << "Data index 0 : ";
26    << characterList.get(0)
27    << endl;
28    characterList.set("Z", 0);
29    cout << "Setelah set data : ";
30    characterList.display();
31    characterList.clear();
32    cout << "Setelah clear : ";
33    characterList.display();
34
35    return 0;
36 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Setelah delete back : ts linked list : A V 0
Setelah delete index 1 : ts linked list : A 0
Data index 0 : A
Setelah set data : ts linked list : Z 0
Setelah clear : linked list kosong
PS C:\Users\USIR\Desktop\Documents\GitHub\module-3-linked-list> g++ .\double_linked_list.cpp
```

Gambar 3 Screenshot Output Program Soal 3

C. Pembahasan

Implementasi DoubleLinkedList pada program menggunakan struktur data *Circular Doubly Linked List*, yaitu linked list yang setiap node memiliki dua hubungan sekaligus, yaitu menuju node berikutnya (next) dan node sebelumnya (prev). Selain itu node terakhir juga kembali terhubung ke node pertama sehingga membentuk struktur melingkar. Seluruh node pada program dialokasikan secara manual di dalam heap menggunakan sintaks `new Node`. Penggunaan heap dipilih karena jumlah data pada linked list bersifat dinamis sehingga ukuran penyimpanan dapat bertambah maupun berkurang selama program berjalan tanpa perlu menentukan kapasitas tetap sejak awal.

Pada setiap proses penambahan data seperti `add_front()`, `add_back()`, dan `add_idx()`, program membuat node baru menggunakan alokasi dinamis. Karena memori dibuat secara manual, maka node yang sudah tidak digunakan wajib dihapus menggunakan `delete` agar tidak terjadi *memory leak*. Oleh sebab itu fungsi penghapusan seperti `delete_front()`, `delete_back()`, `delete_idx()`, dan `clear()` memiliki peran penting untuk membebaskan memori yang sebelumnya telah dialokasikan di heap. Fungsi `clear()` bekerja dengan cara menghapus node satu per satu hingga linked list kosong. Dengan cara ini seluruh memori yang tidak lagi digunakan dapat dikembalikan ke sistem sehingga program menjadi lebih aman dan efisien.

Fungsi `init()` memiliki kompleksitas waktu $O(1)$ karena hanya mengatur nilai head, tail, dan size tanpa melakukan perulangan. Kompleksitas ruangnya juga $O(1)$ karena tidak menggunakan memori tambahan selain variabel yang sudah ada. Fungsi `is_empty()` mempunyai kompleksitas waktu $O(1)$ karena hanya melakukan satu kali pengecekan kondisi pada pointer head. Kompleksitas ruangnya juga tetap $O(1)$. Pada fungsi `add_front()` dan `add_back()`, kompleksitas waktunya adalah $O(1)$ karena penambahan node dilakukan langsung pada bagian depan atau belakang tanpa perlu menelusuri seluruh linked list. Kompleksitas ruangnya adalah $O(1)$ karena hanya membutuhkan satu node baru sebagai tambahan memori. Fungsi `add_idx()` memiliki kompleksitas waktu $O(n)$ karena program harus bergerak menuju posisi index tertentu sebelum menyisipkan node baru. Pada kondisi terburuk, traversal dapat berjalan hingga mendekati seluruh isi

linked list. Kompleksitas ruangnya tetap $O(1)$ karena hanya menggunakan satu node tambahan. Pada fungsi `delete_front()` dan `delete_back()`, kompleksitas waktunya adalah $O(1)$ karena node depan maupun belakang dapat langsung dihapus menggunakan bantuan pointer head dan tail. Kompleksitas ruang kedua fungsi tersebut tetap $O(1)$. Fungsi `delete_idx()` memiliki kompleksitas waktu $O(n)$ karena program harus melakukan traversal menuju posisi node target sebelum node dapat dihapus. Kompleksitas ruangnya tetap $O(1)$ karena tidak membutuhkan struktur data tambahan. Pada fungsi `display()`, kompleksitas waktunya adalah $O(n)$ karena seluruh node harus ditampilkan satu per satu hingga kembali ke head. Kompleksitas ruangnya tetap $O(1)$. Fungsi `get()` dan `set()` juga memiliki kompleksitas waktu $O(n)$ karena program perlu bergerak menuju index tertentu sebelum mengambil atau mengubah data. Kompleksitas ruang kedua fungsi tersebut tetap $O(1)$. Sementara itu fungsi `clear()` memiliki kompleksitas waktu $O(n)$ karena setiap node harus dihapus satu per satu sampai linked list kosong. Kompleksitas ruangnya adalah $O(1)$ karena penghapusan dilakukan langsung tanpa menggunakan struktur data tambahan.

Algoritma yang dirancang pada implementasi ini dapat dianggap valid karena setiap operasi selalu menjaga hubungan dua arah (next dan prev) antar node serta mempertahankan sifat circular antara head dan tail. Ketika node ditambahkan atau dihapus, pointer next dan prev selalu diperbarui sehingga hubungan antar node tetap benar dan tidak terputus. Validasi index pada fungsi seperti `add_idx()`, `delete_idx()`, `get()`, dan `set()` juga membantu mencegah akses data di luar batas linked list. Selain itu penggunaan delete pada setiap proses penghapusan memastikan memori yang sudah tidak digunakan dapat dibersihkan dengan benar sehingga program tidak mengalami kebocoran memori.

SOAL 4

Si Palui dan Tarik Tambang Dimensi

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui mengamati fenomena aneh di atas sebuah gelanggang melingkar yang berisi N karakter. Terdapat dua proyektor anomali, Alpha (dimulai dari head, bergerak maju menggunakan next) dan Beta (dimulai dari tail, bergerak mundur menggunakan prev).

Palui melakukan observasi selama R ronde. Pada setiap ronde:

1. Proyektor Alpha dipaksa melompat maju sebanyak X langkah. Proyektor Beta dipaksa melompat mundur sebanyak Y langkah (simultan).
2. Tabrakan Singular: Jika Alpha dan Beta mendarat di titik atau karakter yang sama persis secara bersamaan, karakter tersebut akan hancur dan lenyap dari lingkaran dimensi. Jika ini terjadi, Alpha berpindah ke karakter **next** dari karakter yang hancur, dan Beta berpindah ke karakter **prev** dari karakter yang hancur.
3. Resonansi Bersebelahan (Adjacent): Jika Alpha dan Beta saling bersebelahan langsung, sifat kuantum dari kedua karakter tersebut saling bertukar (data antara kedua node tersebut di-swap).

Tentukan sisa karakter pada lingkaran dimensi setelah R ronde selesai dieksekusi.

Batasan

- $1 \leq N \leq 10^5$
- $1 \leq R \leq 10^5$
- $1 \leq X, Y \leq 10^{18}$
- Struktur wajib menggunakan implementasi struktur data dari Soal 3.

Format Input

N R

C1 C2 ... CN

X1 Y1

X2 Y2

...

XR YR

Keterangan: C_i adalah satu karakter char tunggal.

Format Output

Karakter yang tersisa, dicetak berurutan mulai dari head hingga ke elemen terakhir, tanpa spasi. Jika dimensi menjadi kosong, cetak EMPTY

CONTOH KASUS

Input	Output
4 2 A B C D 2 3 1 1	BADC
5 3 V W X Y Z 100000 100000 2 2 5 5	XWZY

A. Source Code

Tabel 8 Source Code File prob_b.cpp

1	#include "double_linked_list.h"
2	#include <iostream>
3	
4	using namespace std;
5	void deleteNode(DoubleLinkedList &dll, Node*
6	target) {
7	if (dll.size == 1) {
8	delete target;
9	dll.head = nullptr;
10	dll.tail = nullptr;
11	dll.size = 0;
12	return;
13	}
14	
15	target->prev->next = target->next;
16	target->next->prev = target->prev;
17	
18	if (target == dll.head)
19	dll.head = target->next;
20	
21	if (target == dll.tail)
22	dll.tail = target->prev;
23	
24	delete target;
25	dll.size--;
26	}
27	
28	void swapAdjacent(DoubleLinkedList &dll, Node* a,
	Node* b) {

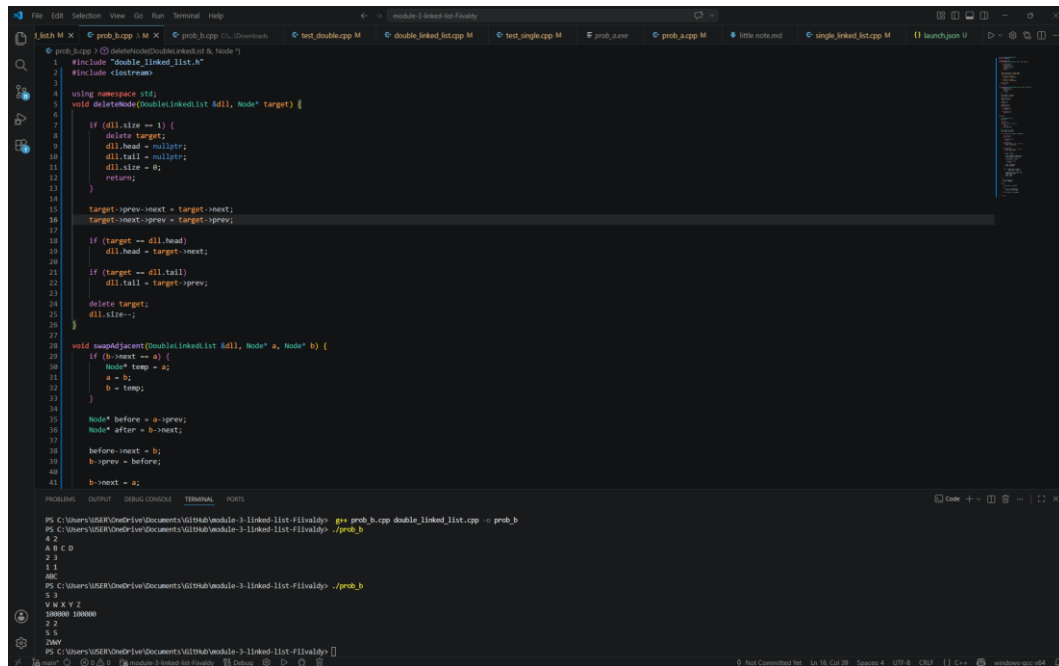
29	if (b->next == a) {
30	Node* temp = a;
31	a = b;
32	b = temp;
33	}
34	
35	Node* before = a->prev;
36	Node* after = b->next;
37	
38	before->next = b;
39	b->prev = before;
40	
41	b->next = a;
42	a->prev = b;
43	
44	a->next = after;
45	after->prev = a;
46	
47	if (dll.head == a)
48	dll.head = b;
49	
50	if (dll.tail == b)
51	dll.tail = a;
52	}
53	
54	int main() {
55	
56	DoubleLinkedList dll;
57	dll.init();
58	
59	int N, R;
60	cin >> N >> R;

61	for (int i = 0; i < N; i++) {
62	char c;
63	cin >> c;
64	dll.add_back(c);
65	}
66	
67	Node* alpha = dll.head;
68	Node* beta = dll.tail;
69	
70	for (int ronde = 0; ronde < R; ronde++) {
71	
72	long long X, Y;
73	cin >> X >> Y;
74	
75	if (dll.size == 0)
76	break;
77	
78	X %= dll.size;
79	for (long long i = 0; i < X; i++) {
80	alpha = alpha->next;
81	}
82	
83	Y %= dll.size;
84	for (long long i = 0; i < Y; i++) {
85	beta = beta->prev;
86	}
87	
88	if (alpha == beta) {
89	
90	Node* nextAlpha = alpha->next;
91	Node* prevBeta = beta->prev;
92	

93	deleteNode(dll, alpha);
94	
95	if (dll.size == 0)
96	break;
97	
98	alpha = nextAlpha;
99	beta = prevBeta;
100	}
101	
102	else if (alpha->next == beta
103	beta->next == alpha) {
104	
105	swapAdjacent(dll, alpha, beta);
106	Node* temp = alpha;
107	alpha = beta;
108	beta = temp;
109	}
110	}
111	
112	if (dll.is_empty()) {
113	cout << "KOSONG";
114	}
115	else {
116	
117	Node* current = dll.head;
118	
119	do {
120	cout << current->data;
121	current = current->next;
122	}
123	while (current != dll.head);
124	}

125	<pre> return 0; }</pre>
126	
127	

B. Output Program



```
1 #include <iostream>
2 #include "double_linked_list.h"
3
4 using namespace std;
5 void deleteNode(DoubleLinkedList &dll, Node* target) {
6     if (dll.size == 1) {
7         delete target;
8         dll.head = nullptr;
9         dll.tail = nullptr;
10        dll.size = 0;
11        return;
12    }
13    target->prev->next = target->next;
14    target->next->prev = target->prev;
15
16    if (target == dll.head)
17        dll.head = target->next;
18    if (target == dll.tail)
19        dll.tail = target->prev;
20
21    delete target;
22    dll.size--;
23 }
24
25 void swapAdjacent(DoubleLinkedList &dll, Node* a, Node* b) {
26     if (a->next == b) {
27         Node* temp = a;
28         a = b;
29         b = temp;
30     }
31
32     Node* before = a->prev;
33     Node* after = b->next;
34
35     before->next = b;
36     b->prev = before;
37     b->next = a;
38     a->prev = after;
39 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL RUNS

```
PS C:\Users\USER\OneDrive\Documents\GitHub\module-3-linked-list-1\invalid> g++ prob_b.cpp double_linked_list.cpp -o prob_b
4 2
A B C D
2 3
1 1
000
5 1
V W X Y Z
100000 100000
2 2
5 5
20000
```

Gambar 4 Output Program Soal 4

C. Pembahasan

Program pada file `prob_b.cpp` digunakan untuk mensimulasikan pergerakan dua proyektor bernama Alpha dan Beta pada sebuah struktur data circular double linked list. Struktur data ini memungkinkan perpindahan node dilakukan ke arah kanan maupun kiri secara melingkar. Pada bagian awal program di baris [1]-[4], program melakukan import library dan menggunakan namespace standar C++. Library "`double_linked_list.h`" digunakan karena seluruh operasi linked list seperti penambahan node dan penghapusan node sudah dibuat pada file tersebut.

Pada baris [6]-[28], dibuat fungsi `deleteNode()` yang berfungsi untuk menghapus node tertentu secara langsung menggunakan pointer. Fungsi ini menerima parameter berupa objek linked list dan node target yang ingin dihapus. Pada kondisi di baris [8]-[14], program memeriksa apakah linked list hanya memiliki satu node. Jika benar, maka node langsung dihapus dan pointer head serta tail dikosongkan. Jika jumlah node lebih dari satu, maka pada baris [16]-[25] dilakukan penyambungan ulang antar node sebelum node target dihapus. Proses ini penting agar linked list tetap tersambung secara melingkar setelah penghapusan node dilakukan.

Selanjutnya pada baris [30]-[53], terdapat fungsi `swapAdjacent()` yang digunakan untuk menukar posisi dua node yang saling bersebelahan. Pada baris [31]-[35], program memastikan bahwa node a selalu berada sebelum node b. Jika ternyata posisi node terbalik, maka pointer ditukar terlebih dahulu. Setelah itu, pada baris [37]-[46], dilakukan pengaturan ulang hubungan pointer next dan prev antar node agar posisi kedua node benar-benar bertukar di dalam linked list. Kemudian pada baris [48]-[52], program memeriksa apakah node yang dipindahkan merupakan head atau tail, sehingga pointer utama linked list juga ikut diperbarui. Fungsi utama program dimulai pada baris [55]. Pada baris [57]-[58], dibuat objek `DoubleLinkedList` bernama `dll` dan dilakukan inisialisasi linked list menggunakan fungsi `init()`. Kemudian pada baris [60]-[61], program membaca input jumlah karakter (N) dan jumlah ronde simulasi (R).

Pada baris [62]-[66], program membaca karakter satu per satu lalu memasukkannya ke bagian belakang linked list menggunakan fungsi `add_back()`.

Dengan cara ini, urutan karakter pada input akan tetap sama seperti data awal yang diberikan pengguna.

Selanjutnya pada baris [68]-[69], ditentukan posisi awal kedua proyektor. Alpha dimulai dari node head, sedangkan Beta dimulai dari node tail. Kedua pointer ini nantinya akan bergerak selama simulasi berlangsung.

Perulangan utama simulasi terdapat pada baris [71]-[107]. Pada setiap ronde, program membaca nilai langkah Alpha (X) dan Beta (Y) pada baris [73]-[74]. Kemudian pada baris [76]-[77], program memeriksa apakah linked list sudah kosong. Jika kosong, maka simulasi dihentikan karena tidak ada lagi node yang dapat diproses. Pergerakan Alpha dilakukan pada baris [79]-[82]. Nilai langkah X terlebih dahulu dimodifikasi menggunakan operasi modulo terhadap ukuran linked list agar perpindahan tetap efisien walaupun jumlah langkah sangat besar. Setelah itu Alpha bergerak maju menggunakan pointer next. Hal yang sama dilakukan untuk Beta pada baris [84]-[87], namun Beta bergerak mundur menggunakan pointer prev.

Pada baris [89]-[101], program memeriksa kemungkinan terjadinya tabrakan. Jika Alpha dan Beta berada pada node yang sama, maka node tersebut akan dihapus menggunakan fungsi deleteNode(). Sebelum penghapusan dilakukan, pointer tujuan selanjutnya disimpan terlebih dahulu pada baris [91]-[92] agar Alpha dan Beta masih dapat melanjutkan simulasi setelah node dihapus. Setelah penghapusan selesai, posisi Alpha dan Beta diperbarui pada baris [99]-[100].

Jika tidak terjadi tabrakan, maka program akan memeriksa apakah Alpha dan Beta berada pada node yang saling bersebelahan pada baris [103]-[104]. Jika kondisi tersebut terpenuhi, maka fungsi swapAdjacent() dipanggil untuk menukar posisi kedua node tersebut. Setelah posisi node bertukar, pointer Alpha dan Beta juga ikut ditukar pada baris [106]-[107] agar proyektor tetap mengikuti node yang sama.

Bagian akhir program terdapat pada baris [110]-[121]. Pada baris [110]-[112], program memeriksa apakah linked list kosong. Jika kosong, maka program menampilkan tulisan "KOSONG". Namun jika masih terdapat node, maka program melakukan traversal mulai dari head pada baris [115]-[120] dan mencetak seluruh karakter secara berurutan hingga kembali lagi ke head. Hasil akhir inilah yang menjadi output dari simulasi program.

